

DOCUMENTAÇÃO DE USO — KILO CODE

Versão: 1.0

Status: Funcional

Última atualização: 2026-05-12

1. REQUISITOS

```
Python 3.10+
sentence-transformers >= 2.2.0
networkx >= 3.0
numpy >= 1.24
scikit-learn >= 1.3
pyyaml >= 6.0
```

Instalação:

```
pip install -e .
# ou, se houver requirements.txt:
pip install -r requirements.txt
```

2. ENTRY POINT

O ponto de entrada único é `run_kilo.py`, localizado na raiz do projeto:

```
e:/Arquivos/Área de Trabalho/MestreCuca/
└─ run_kilo.py          ← SEMPRE use este arquivo
```

⚠ **Não use** `.kilo/agents/driver.py` — possui issues de path no Windows.
`run_kilo.py` resolve todos os paths automaticamente.

3. MODO DE USO

3.1 Health Check

Verifica integridade de todos os componentes:

```
python run_kilo.py --health
```

Saída esperada:

```
[HEALTH] ✓ OntologyGraph: 242 nós, 2557 arestas
[HEALTH] ✓ HybridRetriever: 162 embeddings carregados
[HEALTH] ✓ Classifier: 54 subárvores mapeadas
[HEALTH] ✓ Validator: 162 células no registry
[HEALTH] ✓ Synthesizer: 5 templates de pilar carregados
[HEALTH] ✓ Router: 6 pilares configurados
...
✓ 13/13 verificações passaram
```

3.2 Query Simples

```
python run_kilo.py --query "como resolver conflitos?"
```

Usa o pipeline **simple** (router → retriever → synthesis → validator).

3.3 Pipeline Completo

```
python run_kilo.py --query "como otimizar um processo de produção?" --mode full
```

Usa o pipeline **full** (classifier → router → retriever → graph → dialectic → synthesis → validator).

3.4 Modo Interativo

```
python run_kilo.py --mode interactive
```

Abre um loop REPL onde você faz queries sucessivas sem recarregar o modelo.

3.5 Modo Autônomo

```
python run_kilo.py --query "..." --mode autonomous
```

Executa o pipeline completo com **AutonomyLayer** ativo:

- Auto-avaliação de qualidade
- Feedback loop
- Thresholds adaptativos
- Memória episódica

3.6 Debug / Trace

```
python run_kilo.py --query "... " --mode full --verbose --save-trace
```

- `--verbose`: logging detalhado de cada etapa
- `--save-trace`: salva trace completo em `data/traces/`

4. API PROGRAMÁTICA

4.1 Importação

```
import sys
sys.path.insert(0, ".")

from runtime.orchestrator import MultiAgentOrchestrator
```

4.2 Inicialização

```
orchestrator = MultiAgentOrchestrator(
    config={
        "json_dir": "data/json",
        "embeddings_dir": "data/embeddings",
        "config_dir": ".kilo/config",
        "index_path": "data/json/ontology_index.json",
    }
)
```

O construtor executa `_warm_up()` automaticamente, carregando o modelo de embeddings e fazendo uma query de teste.

4.3 Execução do Pipeline

```
# Pipeline completo
result = orchestrator.full_pipeline("como lidar com conflito emocional?")

# Pipeline simplificado
result = orchestrator.simple_pipeline("o que é decomposição?")

# Pipeline customizado
result = orchestrator.run_pipeline(
    "como otimizar processos?",
    steps=["router", "retriever", "synthesis", "validator"]
)
```

4.4 Resultado

```

# Como dicionário
data = result.to_dict()

# Campos disponíveis:
# - result.query
# - result.pipeline_mode
# - result.status ("success", "partial", "failed")
# - result.total_latency_ms
# - result.steps: List[PipelineStepResult]
# - result.final_output: Dict[str, Any]
#   - final_output["prompt"]: prompt gerado
#   - final_output["validation"]: resultado da validação
#   - final_output["classification"]: classificação ontológica
#   - final_output["retrieval"]: células recuperadas
#   - final_output["dialectic_context"]: opostos e tensões

# Sumário legível
print(result.summary())

```

4.5 Uso Direto dos Módulos

```

# Classificador standalone
from runtime.classifier import OntologicalClassifier

clf = OntologicalClassifier(json_dir="data/json")
classification = clf.classify("como resolver problemas complexos?")
print(classification.to_dict())

# Retrieval standalone
from runtime.retriever import HybridRetriever

retriever = HybridRetriever(
    embedding_dir="data/embeddings",
    json_dir="data/json",
    config_path=".kilo/config/retrieval.yaml"
)
results = retriever.hybrid_search("como resolver conflitos?", top_k=5)

# Grafo standalone
from core.ontology_graph import OntologyGraph

graph = OntologyGraph(json_dir="data/json")
graph.load_registry()
graph.build_graph()

vizinhos = graph.query_neighbors("N4_DECOMPOSICAO", depth=2)
caminhos = graph.semantic_walk("N4_DECOMPOSICAO", "N4_HOLISMO")

# Validator standalone
from runtime.validator import OntologyValidator

```

```
validator = OntologyValidator(json_dir="data/json")
report = validator.validate(routing, response_context)
print(report) # formato legível
print(report.to_dict()) # formato estruturado

# Síntese standalone
from runtime.synthesizer import OntologySynthesizer, SynthesisInput

syn = OntologySynthesizer(json_dir="data/json")
input_data = SynthesisInput(
    query="como otimizar processos?",
    routing=routing_decision,
    retrieved_cells=cells,
    graph_context=graph_data,
    classification=classification_data,
)
output = syn.synthesize(input_data)
print(output.response)
```

4.6 Agentes Individuais

```
from agents.classifier_agent import ClassifierAgent
from agents.graph_agent import GraphAgent
from agents.synthesis_agent import SynthesisAgent
from agents.validator_agent import ValidatorAgent
from agents.dialectic_agent import DialecticAgent

# Cada agente encapsula seu módulo com caching e tratamento de erros
classifier = ClassifierAgent()
result = classifier.execute("como decompor um problema?")
print(result.to_dict())
```

5. CONFIGURAÇÃO

5.1 Parâmetros Principais

config/retrieval.yaml:

```
vetorial:
  modelo:
    nome: "sentence-transformers/all-MiniLM-L6-v2"
  limiar_minimo: 0.15
  top_k: 20

grafico:
  profundidade_max: 3
```

```
fusao:
  pesos:
    vetorial: 0.50
    grafico: 0.30
    simbolico: 0.20
  top_k_saida: 12
  limiar_final: 0.40
```

config/ontology.yaml:

```
naturezas_permitidas:
- processo
- estado
- fenomeno
- principio
- mecanismo
- estrutura
- arquetipo
- dinamica
- restricao
- vetor

tipos_relacao:
  depende_de: 1.0
  complementa: 0.8
  contrasta: -0.7
  expande: 0.6
  implementa: 0.7
  generaliza: 0.6
  especializa: 0.6
  causa: 0.7
  equilibra: 0.5
  transforma: 0.6
```

5.2 Thresholds Recomendados

Parâmetro	Valor	Efeito
semantic_threshold	0.72	Filtra embeddings fracos
graph_depth	3	Profundidade de expansão no grafo
rerank_limit	12	Número de resultados finais
validation_threshold	0.60	Score mínimo para aprovação

6. FORMATO DOS DADOS

6.1 JSON de uma Célula N4

```
{
  "uid": "N4_DECOMPOSICAO",
  "path": "S1.L1.1.1-A",
  "nome": "Decomposição",
  "nome_normalizado": "decomposicao",
  "descricao": "Processo de dividir um problema complexo...",
  "natureza": "processo",
  "funcao_cognitiva": ["decompor", "segmentar", "modularizar"],
  "assinatura_semantica": {
    "abstracao": 0.82,
    "complexidade": 0.71,
    "causalidade": 0.65,
    "emocionalidade": 0.12,
    "materialidade": 0.34,
    "simbolismo": 0.15,
    "dinamismo": 0.78,
    "temporalidade": 0.55,
    "ambiguidade": 0.08
  },
  "relacoes": [
    {"tipo": "depende_de", "alvo": "N4_ANALISE", "peso": 0.82},
    {"tipo": "complementa", "alvo": "N4_INTEGRACAO", "peso": 0.71}
  ],
  "opostos": ["N4_HOLISMO"],
  "tags": ["analise", "estrutura", "metodo"],
  "heranca": {
    "n0": "SINTRÓPICO",
    "n1": "LOGOS",
    "n2": "ALGORITMIA",
    "n3": "RESOLUÇÃO"
  }
}
```

6.2 Resultado do Pipeline

```
{
  "query": "como resolver conflitos?",
  "pipeline_mode": "full",
  "status": "success",
  "total_latency_ms": 3450.2,
  "steps": [
    {
      "step": 1,
      "agent_name": "classifier",
      "success": true,
      "latency_ms": 230.5,
      "confidence": 0.89,
      "reasoning": ["SINTRÓPICO via keywords: resolver, conflito", "LOGOS via dependência estrutural"]
    },
  ],
}
```

```
{
  "step": 2,
  "agent_name": "router",
  "success": true,
  "latency_ms": 15.2,
  "confidence": 0.92
},
{
  "step": 3,
  "agent_name": "retriever",
  "success": true,
  "latency_ms": 180.3,
  "retrieved_count": 12
},
{
  "step": 4,
  "agent_name": "graph",
  "success": true,
  "latency_ms": 45.1,
  "neighbors_found": 28
},
{
  "step": 5,
  "agent_name": "dialectic",
  "success": true,
  "latency_ms": 120.8,
  "opposites_found": 3
},
{
  "step": 6,
  "agent_name": "synthesis",
  "success": true,
  "latency_ms": 2100.0,
  "prompt_length": 4520
},
{
  "step": 7,
  "agent_name": "validator",
  "success": true,
  "latency_ms": 85.4,
  "score_geral": 0.87,
  "aprovado": true
}
],
"final_output": {
  "prompt": "...",
  "validation": {
    "aprovado": true,
    "score_geral": 0.87,
    "criticidade": "media"
  },
  "classification": {
    "eixo": "SINTRÓPICO",
    "pilar": "PATHOS",
```



```

    "dominio": "ALTERIDADE"
  }
}

```

7. EXEMPLOS DE QUERIES

Query	Pipeline	Classificação Esperada
"como otimizar um algoritmo?"	full	LOGOS > ALGORITMIA > OTIMIZAÇÃO
"como lidar com perda?"	full	PATHOS > ETHOS > VALORES
"o que é entropia?"	simple	KHAOS > ENTROPIA > DEGRADAÇÃO
"como escalar um sistema?"	full	APEIRON > ESCALA > PROPORÇÃO
"qual o sentido da vida?"	full	MYTHOS > MISTERIO > REVELAÇÃO
"como funciona um ecossistema?"	simple	BIOS > OIKOS > MORADA

8. TROUBLESHOOTING

Erro: `ModuleNotFoundError: No module named 'kilo'`

Causa: `run_kilo.py` não está sendo executado a partir do diretório correto.

Solução:

```

cd /caminho/para/MestreCuca
python run_kilo.py --health

```

Erro: `FileNotFoundError: data/json/ontology_index.json`

Causa: Índice não foi gerado.

Solução:

```

python tools/n4_to_json.py
# ou
python tools/enrich_json.py

```

Erro: `Nenhum embedding encontrado`

Causa: Diretório `data/embeddings/` vazio ou arquivos `.npy` ausentes.

Solução:

```
python tools/build_embeddings.py
```

Erro: Health check falha no grafo

Causa: `data/graphs/ontology.gexf` pode estar corrompido.

Solução:

```
python tools/graph_builder.py --rebuild
```

Performance lenta na primeira query

Causa: Modelo sentence-transformers sendo carregado (~10s).

Mitigação:

```
# Pré-carregar explicitamente
from runtime.retriever import HybridRetriever
retriever = HybridRetriever() # carrega modelo aqui
# Agora o pipeline será mais rápido
```

9. ESTRUTURA DE TESTES

```
tests/
├── test_classifier.py      # Classificação N0→N4
├── test_retriever.py      # Retrieval híbrido
├── test_graph.py          # Grafo ontológico
├── test_synthesizer.py    # Síntese de respostas
├── test_validator.py      # Validação ontológica
├── test_orchestrator.py   # Pipeline completo
├── test_agents.py         # Agentes individuais
├── test_autonomy.py       # AutonomyLayer
└── test_integration.py    # End-to-end
```

Execute:

```
python -m pytest tests/ -v
```

10. ARQUITETURA DO SISTEMA

Consulte [docs/arquitetura_sistema_cognitivo.md](#) para detalhes completos sobre:

- Topologia do grafo (242 nós, ~2557 arestas)
- Modelo de assinaturas semânticas 9D
- Tipagem ontológica (10 naturezas)
- Tipos de relação (10 tipos com pesos)
- Padrões de projeto aplicados
- Decisões arquiteturais e justificativas
- Roadmap de extensões futuras

Documentação de uso — Kilo Code v1.0